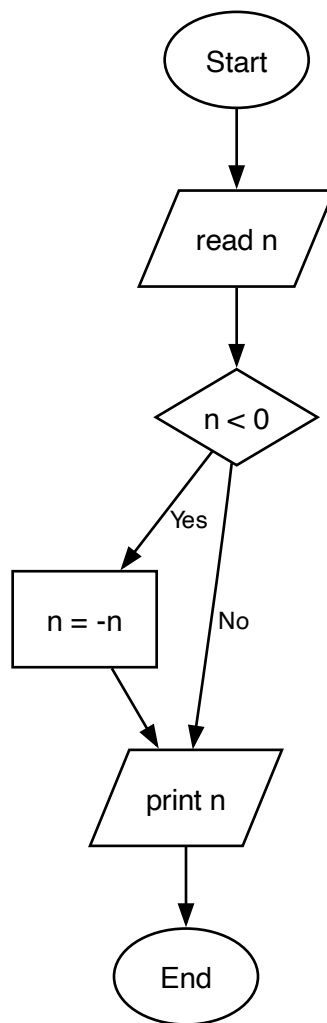


Answer key

Problem 1.



(i) For each input below, follow the flowchart and write the output.

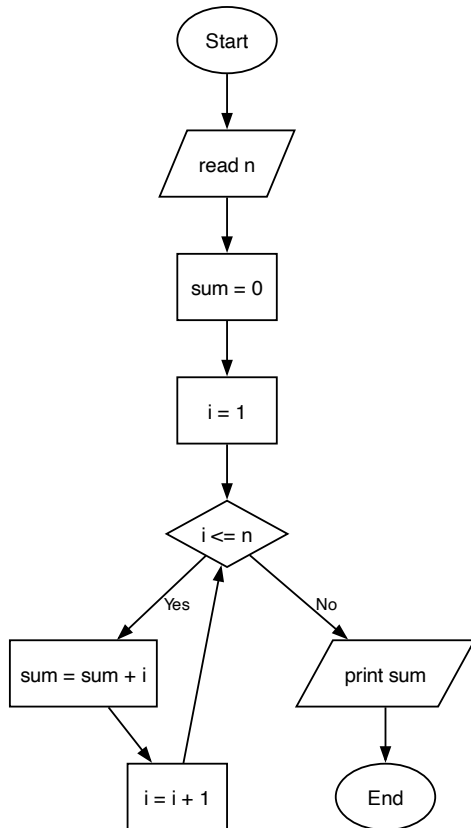
input n	output
7	7
-4	4
0	0
-19	19

(ii) Can you tell what this algorithm is doing?

Answer: Absolute value

Problem 2.

Trace this algorithm for $n = 4$, and complete the trace table.

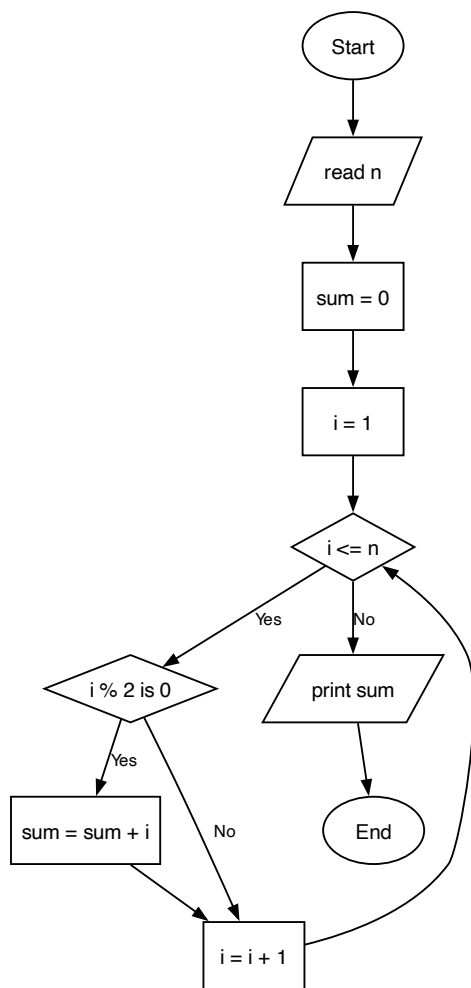


n	sum	i	output
4			
4	0		
4	0	1	
4	1	1	
4	1	2	
4	3	2	
4	3	3	
4	6	3	
4	6	4	
4	10	4	
4	10	5	
4	10	5	10

Output: 10

Problem 3.

Trace this algorithm for $n = 5$, and complete the trace table.

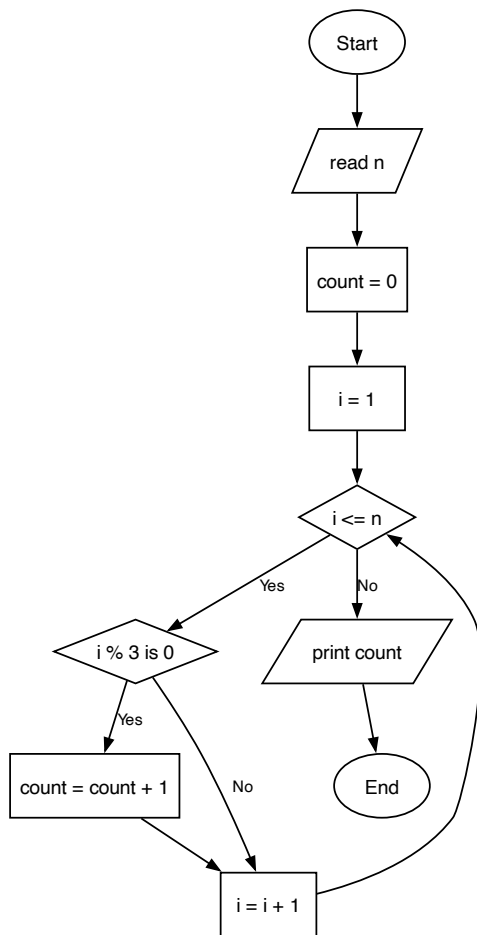


n	sum	i	output
5			
5	0		
5	0	1	
5	0	2	
5	2	2	
5	2	3	
5	2	4	
5	6	4	
5	6	5	
5	6	6	
5	6	6	6

Output: 6

Problem 4.

Trace this algorithm for $n = 7$, and complete the trace table.



n	count	i	output
7			
7	0		
7	0	1	
7	0	2	
7	0	3	
7	1	3	
7	1	4	
7	1	5	
7	1	6	
7	2	6	
7	2	7	
7	2	8	
7	2	8	2

Output: 2

Problem 5.

Trace this algorithm for $n = 472$, and complete the trace table.

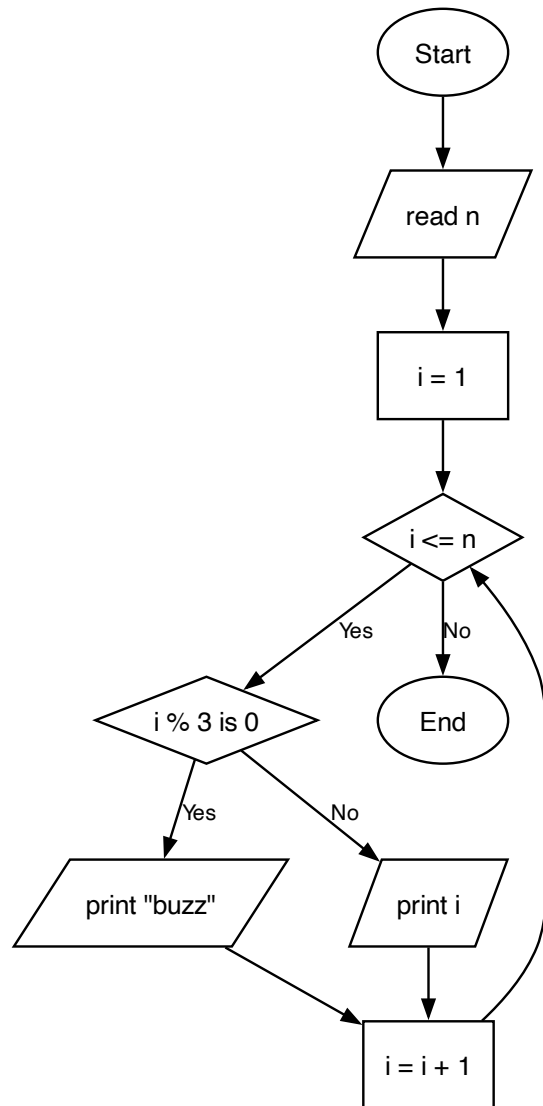
Note: $n // 10$ means whole-number division — divide and drop the remainder. For example, $472 // 10 = 47$ (not 47.2).



n	count	output
472		
472	0	
472	1	
47	1	
47	2	
4	2	
4	3	
0	3	
0	3	3

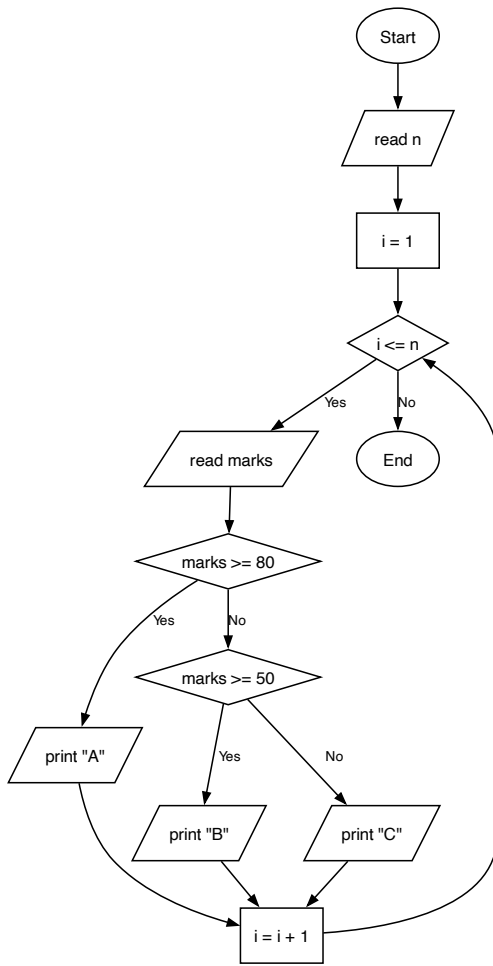
Output: 3

Problem 6.



Problem 7.

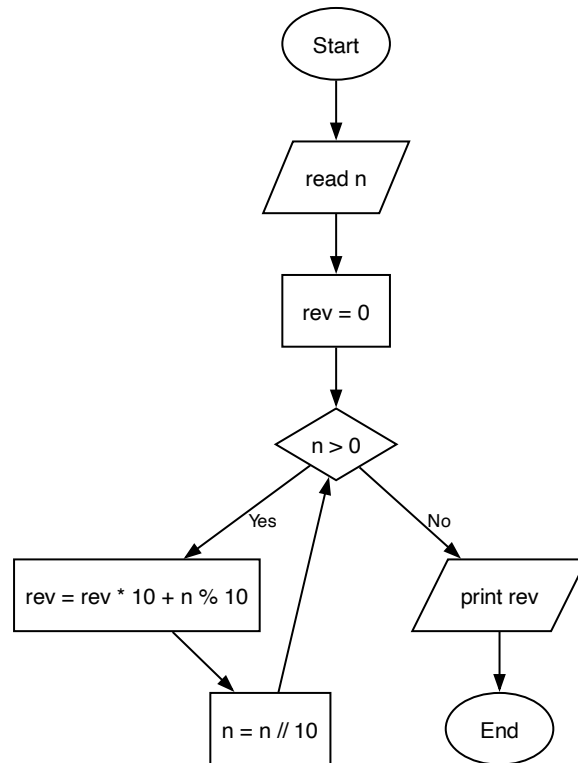
Trace this algorithm for $n = 3$, where the values of *marks* entered are 55, 82, 40 (in that order), and complete the trace table.



n	i	marks	output
3			
3	1		
3	1	55	
3	1	55	B
3	2	55	
3	2	82	
3	2	82	A
3	3	82	
3	3	40	
3	3	40	C
3	4	40	

Output: B, A, C

Problem 8.



(i) For each input below, follow the flowchart and write the output.

Note: $n \% 10$ gives the remainder after dividing by 10 — the last digit of n . For example, $472 \% 10 = 2$.

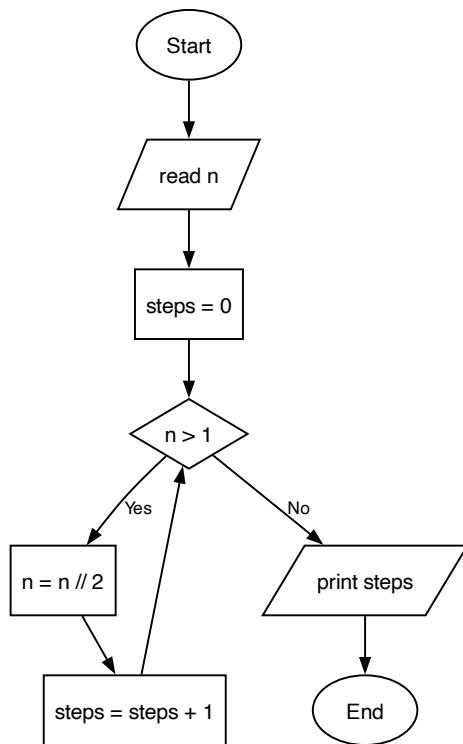
input n	output
123	321
47	74
5	5
470	74

(ii) Can you tell what this algorithm is doing?

Answer: Reverse the digits

Problem 9.

(i) Trace this algorithm for $n = 16$, and complete the trace table.



n	steps	output
16		
16	0	
8	0	
8	1	
4	1	
4	2	
2	2	
2	3	
1	3	
1	4	
1	4	4

(ii) For each input below, follow the flowchart and write the output.

input n	output
8	3
16	4
100	6
1000	9

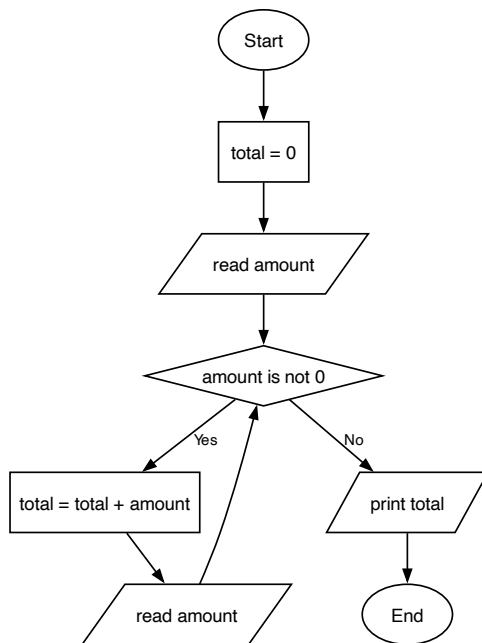
(iii) n grew from 8 all the way to 1000 — but look how little the output grew! Can you explain why?

Answer: Counting halvings — doubling n adds only ONE step. This is the secret behind binary search!

Problem 10.

At the end of the day, a shopkeeper adds up the money received from each customer. She enters the amounts one by one, and enters 0 when there are no more. The program then shows the total. Notice that the algorithm cannot know in advance how many customers there were!

Trace this algorithm, where the values of *amount* entered are 120, 75, 200, 0 (in that order), and complete the trace table.



total	amount	output
0		
0	120	
120	120	
120	75	
195	75	
195	200	
395	200	
395	0	
395	0	395

Output: 395

Problem 11.

